



云计算

CLOUD COMPUTING Third Edition

第2章

Google云计算原理与应用（一）

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎



Google拥有全球最大搜索引擎，除了搜索业务Google还有Google Maps、Google Earth、Gmail、YouTube等其他业务。这些应用的共性在于数据量巨大，且要面向全球用户提供实时服务。

如何解决海量数据存储和快速处理问题？

Google研发出了简单而又高效的技术，让多达百万台的廉价计算机协同工作，共同完成这些任务，这些技术在诞生几年后才被命名为Google云计算技术。

Google云计算技术包括:Google文件系统GFS、分布式计算编程模型MapReduce、分布式锁服务Chubby、分布式结构化数据表Bigtable、分布式存储系统Megastore、分布式监控系统Dapper、海量数据的交互式分析工具Dremel，以及内存大数据分析系统PowerDrill等。

本章详细介绍这八种核心技术和Google应用程序引擎。

2.1 Google文件系统GFS

Google文件系统(GoogleFileSystem, GFS)是一个大型的分布式文件系统。它为 Google云计算提供海量存储, 并且与Chubby、MapReduce及Bigtable等技术结合十分紧密, 处于所有核心技术的底层。GFS不是一个开源的系统, 我们仅能从Google公布的技术文档来获得相关知识。

GoogleGFS的新颖之处在于它采用廉价的商用机器构建分布式文件系统, 同时将 GFS的设计与Google应用的特点紧密结合, 简化实现, 使之可行, 最终达到创意新颖、有用、可行的完美组合。GFS 将容错的任务交给文件系统完成, 利用软件的方法解决系统可靠性问题, 使存储的成本成倍下降。GFS 将服务器故障视为正常现象, 并采用多种方法, 从多个角度, 使用不同的容错措施, 确保数据存储的安全、保证提供不间断的数据存储服务。

2.1 Google文件系统GFS

- ▶ 2.1.1 系统架构
- 2.1.2 容错机制
- 2.1.3 系统管理技术

2.1.1 系统架构

GFS将整个系统的节点分为三类角色:Client(客户端)、Master(主服务器)和Chunk Server(数据块服务器)。

Client是GFS提供给应用程序的访问接口，它是一组专用接口，不遵守 POSIX 规范，以库文件的形式提供。应用程序直接调用这些库函数，并与该库链接在一起。

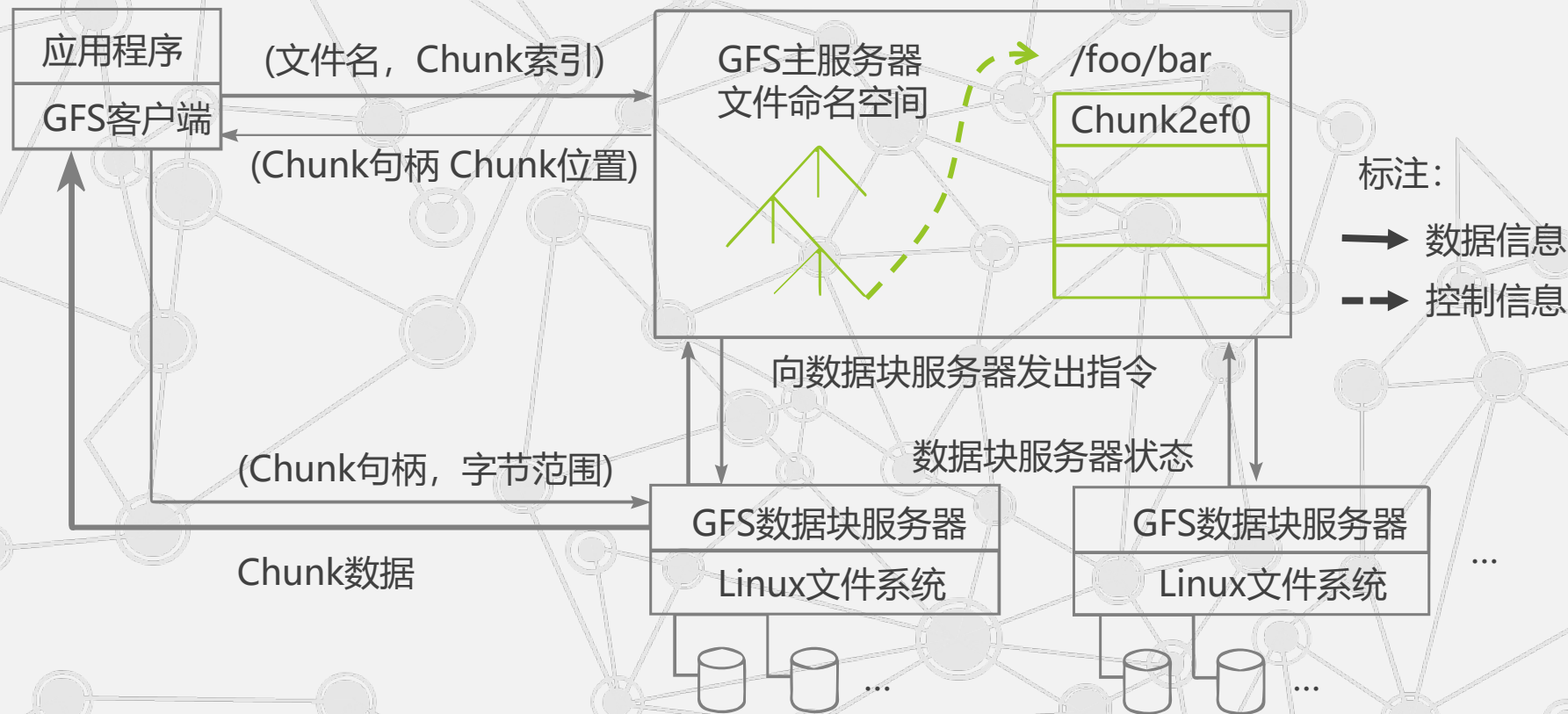
Master是GFS的管理节点，在逻辑上只有一个，它保存系统的元数据，负责整个文件系统的管理，是GFS文件系统中的“大脑”。

Chunk Server负责具体的存储工作。数据以文件的形式存储在Chunk Server上，Chunk Server的个数可以有多个，它的数目直接决定了GFS的规模。

GFS将文件按照固定大小进行分块，默认是64MB，每一块称为一个Chunk(数据块)，每个 Chunk都有一个对应的索引号(Index)。

2.1 Google文件系统GFS

• GFS的系统架构



2.1 Google文件系统GFS

- GFS将整个系统节点分为三类角色



2.1 Google文件系统GFS

● GFS的实现机制

- 客户端首先访问Master节点，获取交互的Chunk Server信息，然后访问这些Chunk Server，完成数据存取工作。这种设计方法实现了控制流和数据流的分离。
- Client与Master之间只有控制流，而无数据流，极大地降低了Master的负载。
- Client与Chunk Server之间直接传输数据流，同时由于文件被分成多个Chunk进行分布式存储，Client可以同时访问多个Chunk Server，从而使得整个系统的I/O高度并行，系统整体性能得到提高。

2.1 Google文件系统GFS

- GFS的特点

采用中心服务器模式

1 GFS 采用**中心服务器模式**管理整个文件系统，简化了设计，降低了实现难度。

Master 管理分布式文件系统中的所有元数据。文件被划分为Chunk进行存储，对于 Master来说，每个Chunk Server只是一个存储空间。Client发起的所有操作都需要先通过Master才能执行。

2.1 Google文件系统GFS

- GFS的特点

采用中心服务器模式

- 可以方便地增加Chunk Server

Chunk Server只需要注册到Master上即可，ChunkServer之间无任何关系。

如果采用完全对等的、无中心的模式，那么如何将ChunkServer的更新信息通知到每一个 Chunk Server，会是设计的一个难点，而这也将在一定程度上影响系统的扩展性。

2.1 Google文件系统GFS

- GFS的特点

采用中心服务器模式

- Master掌握系统内所有Chunk Server的情况，方便进行负载均衡

Master维护了一个统一的命名空间，同时掌握整个系统内ChunkServer的情况，据此可以实现整个系统范围内数据存储的负载均衡。

2.1 Google文件系统GFS

- GFS的特点

采用中心服务器模式

- 不存在元数据的一致性问题

1 由于只有一个中心服务器，元数据的一致性问题自然解决。当然，中心服务器模式也带来一些固有的缺点，比如极易成为整个系统的瓶颈等。

GFS 采用多种机制来避免 Master 成为系统性能和可靠性上的瓶颈，如尽量控制元数据的规模、对 Master 进行远程备份、控制信息和数据分流等。

2.1 Google文件系统GFS

- GFS的特点

不缓存数据

缓存(Cache)机制是提升文件系统性能的一个重要手段，通用文件系统为了提高性能，一般要实现复杂的缓存机制。

GFS文件系统根据应用的特点，没有实现缓存，这是从**必要性和可行性**两方面考虑的。

2.1 Google文件系统GFS

- GFS的特点

不缓存数据

- 文件操作大部分是流式读写，不存在大量重复读写，使用Cache对性能提高不大

从必要性上讲，客户端大部分是流式顺序读写，并不存在大量的重复读写，缓存这部分数据对提高系统整体性能的作用不大：对于ChunkServer，由于GFS的数据在ChunkServer上以文件的形式存储，如果对某块数据读取频繁，本地的文件系统自然会将其缓存。

2.1 Google文件系统GFS

- GFS的特点

不缓存数据

- Chunk Server上数据存取使用本地文件系统从可行性看，Cache与实际数据的一致性维护也极其复杂

从可行性上讲，如何维护缓存与实际数据之间的一致性是一个极其复杂的问题，在GFS中各个ChunkServer的稳定性都无法确保，加之网络等多种不确定因素，一致性问题尤为复杂。此外由于读取的数据量巨大，以当前的内存容量无法完全缓存。对于存储在 Master中的元数据，GFS采取了缓存策略。因为一方面 Master需要频繁操作元数据，把元数据直接保存在内存中，提高了操作的效率。另一方面采用相应的压缩机制降低元数据占用空间的大小，提高内存的利用率。

2.1 Google文件系统GFS

- GFS的特点

在用户态下实现

3 文件系统是操作系统的重要组成部分，通常位于操作系统的底层(内核态)。在内核态实现文件系统，可以更好地和操作系统本身结合，向上提供兼容的POSIX 接口。然而，GFS 却选择在用户态下实现，主要基于以下考虑。

2.1 Google文件系统GFS

● GFS的特点

在用户态下实现

- 
- 利用POSIX编程接口存取数据降低了实现难度，提高通用性
 - POSIX接口提供功能更丰富
 - 用户态下有多种调试工具
 - Master和Chunk Server都以进程方式运行，单个进程不影响整个操作系统
 - GFS和操作系统运行在不同的空间，两者耦合性降低

2.1 Google文件系统GFS

- GFS的特点

在用户态下实现

- 利用POSIX编程接口存取数据降低了实现难度，提高通用性

在用户态下实现，直接利用操作系统提供的POSIX编程接口就可以存取数据，无须了解操作系统的内部实现机制和接口，降低了实现的难度，提高了通用性。

2.1 Google文件系统GFS

- GFS的特点

在用户态下实现

- POSIX接口提供功能更丰富

POSIX接口提供的功能更为丰富，在实现过程中可以利用更多的特性，而不像内核编程那样受限。

2.1 Google文件系统GFS

- GFS的特点

在用户态下实现

- 用户态下有多种调试工具

用户态下有多种调试工具，而在内核态中调试相对比较困难。

2.1 Google文件系统GFS

- GFS的特点

在用户态下实现

- Master和Chunk Server都以进程方式运行，
单个进程不影响整个操作系统

用户态下，Master和Chunk Server都以进程的方式运行，单个进程不会影响到整个操作系统，从而可以对其进行充分优化。在内核态下，如果不能很好地掌握其特性，效率不但不会高，甚至还会影响到整个系统运行的稳定性。

2.1 Google文件系统GFS

- GFS的特点

在用户态下实现

- GFS和操作系统运行在不同的空间，
两者耦合性降低

用户态下，GFS和操作系统运行在不同的空间，两者耦合性降低，方便 GFS自身和内核的单独升级。

2.1 Google文件系统GFS

- GFS的特点

只提供专用接口

4 通常的分布式文件系统一般都会提供一组与 POSIX 规范兼容的接口，使应用程序可以通过操作系统的统一接口透明地访问文件系统，而不需要重新编译程序。

GFS在设计之初，是完全面向Google的应用的，采用了专用的文件系统访问接口。接口以库文件的形式提供，应用程序与库文件一起编译，Google应用程序在代码中通过调用这些库文件的API，完成对GFS文件系统的访问。采用专用接口有以下好处。

2.1 Google文件系统GFS

- GFS的特点

只提供专用接口

- 降低实现难度
- 根据应用特点提供特殊支持
- 直接交互，提高效率

4

2.1 Google文件系统GFS

- GFS的特点

4

只提供专用接口

- 降低实现难度

通常与 POSIX兼容的接口需要在操作系统内核一级实现，而GFS是在应用层实现的。

2.1 Google文件系统GFS

- GFS的特点

4

只提供专用接口

- 根据应用特点提供特殊支持

采用专用接口可以根据应用的特点对应用提供一些特殊支持，如支持多个文件并发追加的接口等。

2.1 Google文件系统GFS

- GFS的特点

4

只提供专用接口

- 直接交互，提高效率

专用接口直接和ClientMaster、ChunkServer交互，减少了操作系统之间上下文的切换，降低了复杂度，提高了效率。

2.1 Google文件系统GFS

2.1.1 系统架构

► 2.1.2 容错机制

2.1.3 系统管理技术

2.1 Google文件系统GFS

- Master容错

Master

命名空间 (Name Space) , 也就是整个文件系统的目录结构。

Chunk与文件名的映射表。

Chunk副本的位置信息, 每一个Chunk默认有三个副本。

日志

直接保存在各个
Chunk Server上

当Master发生故障时, 在磁盘数据保存完好的情况下, 可以迅速恢复以上元数据

为了防止Master彻底死机的情况, GFS还提供了Master远程的实时备份

2.1 Google文件系统GFS

• Chunk Server容错

GFS采用副本的方式实现Chunk Server的容错

每一个Chunk有多个存储副本（默认为三个）

对于每一个Chunk，必须将所有的副本全部写入成功，才视为成功写入

相关的副本出现丢失或不可恢复等情况，Master自动将该副本复制到其他Chunk Server

GFS中的每一个文件被划分成多个Chunk，Chunk的默认大小是**64MB**

每一个Chunk以Block为单位进行划分，大小为64KB，每一个Block对应一个32bit的校验和

2.1 Google文件系统GFS

2.1.1 系统架构

2.1.2 容错机制

► 2.1.3 系统管理技术

2.1.3 系统管理技术

GFS是一个分布式文件系统，包含从硬件到软件的整套解决方案。除了之前提到的GFS的一些关键技术外，还有相应的系统管理技术来支持整个GFS的应用，这些技术可能不一定为GFS独有。

2.1 Google文件系统GFS

● 系统管理技术

系统 管理技术

GFS集群中通常有非常多的节点，需要相应的技术支撑

大规模集群安装技术

故障检测技术

GFS构建在不可靠廉价计算机之上的文件系统，由于节点数目众多，故障发生十分频繁

新的Chunk Server加入时，只需裸机加入，大大减少GFS维护工作量

节点动态加入技术

节能技术

Google采用了多种机制降低服务器能耗，如采用蓄电池代替昂贵的UPS

2.1 Google文件系统GFS

● 系统管理技术

系统 管理技术

GFS集群中通常有非常多的节点，需要相应的技术支撑

大规模集群安装技术

故障检测技术

GFS构建在不可靠廉价计算机之上的文件系统，由于节点数目众多，故障发生十分频繁

新的Chunk Server加入时，只需裸机加入，大大减少GFS维护工作量

节点动态加入技术

节能技术

Google采用了多种机制降低服务器能耗，如采用蓄电池代替昂贵的UPS

2.1 Google文件系统GFS

- 系统管理技术

大规模
集群安装
技术

安装GFS的集群中通常有非常多的节点，而现在的Google数据中心动辄有万台以上的机器在运行。

因此迅速地安装、部署一个 GFS的系统，以及迅速地进行节点的系统升级等，都需要相应的技术支撑。

2.1 Google文件系统GFS

- 系统管理技术

故障检测
技术

GFS 是构建在不可靠的廉价计算机之上的文件系统，由于节点数目众多，故障发生十分频繁，如何在最短的时间内发现并确定发生故障的 Chunk Server，需要相关的集群监控技术。

2.1 Google文件系统GFS

- 系统管理技术

当有新的Chunk Server加入时，如果需要事先安装好系统，那么系统扩展将是一件十分烦琐的事情。

如果能够做到只需将裸机加入，就会自动获取系统并安装运行，那么将会大大减少GFS维护的工作量。

节点
动态
加入技术

2.1 Google文件系统GFS

- 系统管理技术

有关数据表明，服务器的耗电成本大于当初的购买成本，因此Google采用了多利机制来降低服务器的能耗，例如对服务器主板进行修改，采用蓄电池代替昂贵的UPS(不间断电源系统)，提高能量的利用率。Rich Miller在一篇关于数据中心的博客文章中表示，这个设计让Google的UPS利用率达到99.9%，而一般数据中心只能达到92%~95%。

节能
技术

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎

2.2分布式数据处理MapReduce

- ▶ 2.2.1 产生背景
- 2.2.2 编程模型
- 2.2.3 实现机制
- 2.2.4 案例分析

2.2 分布式数据处理 MapReduce

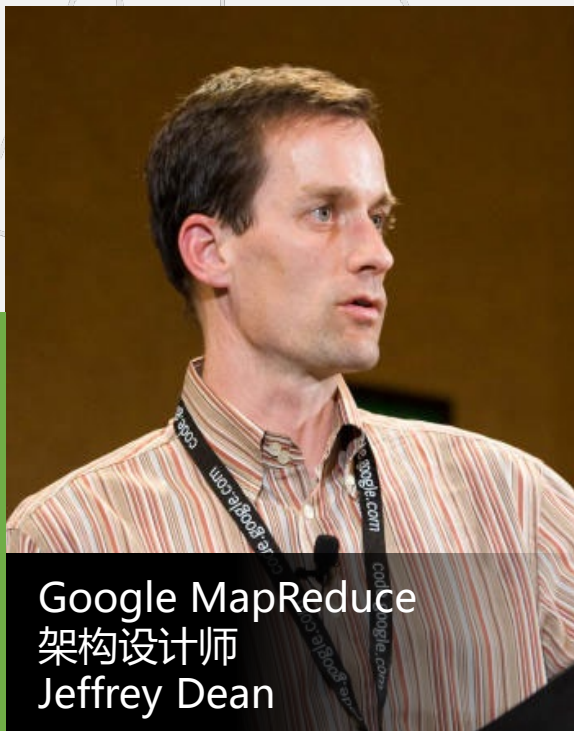
MapReduce是Google提出的一个软件架构，是一种处理海量数据的并行编程模式，用于大规模数据集(通常大于1TB)的并行运算。

Map(映射)Reduce(化简)的概念和主要思想，都是从函数式编程语言和矢量编程语言借鉴来的。

正是由于 MapReduce有函数式和矢量编程语言的共性，使得这种编程模式特别适合于非结构化和结构化的海量数据的搜索、挖掘、分析与机器学习等。

2.2 分布式数据处理 MapReduce

- 产生背景



Jeffery Dean设计一个新的抽象模型，封装并行处理、容错处理、本地化计算、负载均衡的细节，还提供了一个简单而强大的接口。

这就是MapReduce

2.2 分布式数据处理 MapReduce

● 产生背景

MapReduce这种**并行编程模式**思想最早是在**1995年**提出的。

与传统的分布式程序设计相比，MapReduce封装了**并行处理、容错处理、本地化计算、负载均衡**等细节，还提供了一个**简单而强大的接口**。

MapReduce把对数据集的大规模操作，**分发给一个主节点管理下的各分节点**共同完成，通过这种方式**实现任务的可靠执行与容错机制**。

2.2 分布式数据处理 MapReduce

● 产生背景

据相关统计，每使用一次Google搜索引擎，Google的后台服务器就要进行 10^{11} 次运算。这么庞大的运算量，如果没有好的负载均衡机制，有些服务器的利用率会很低，有些则会负荷太重，有些甚至可能死机，这些都会影响系统对用户的服务质量。

而使用MapReduce这种编程模式，就保持了服务器之间的均衡，提高了整体效率。

2.2分布式数据处理MapReduce

2.2.1 产生背景

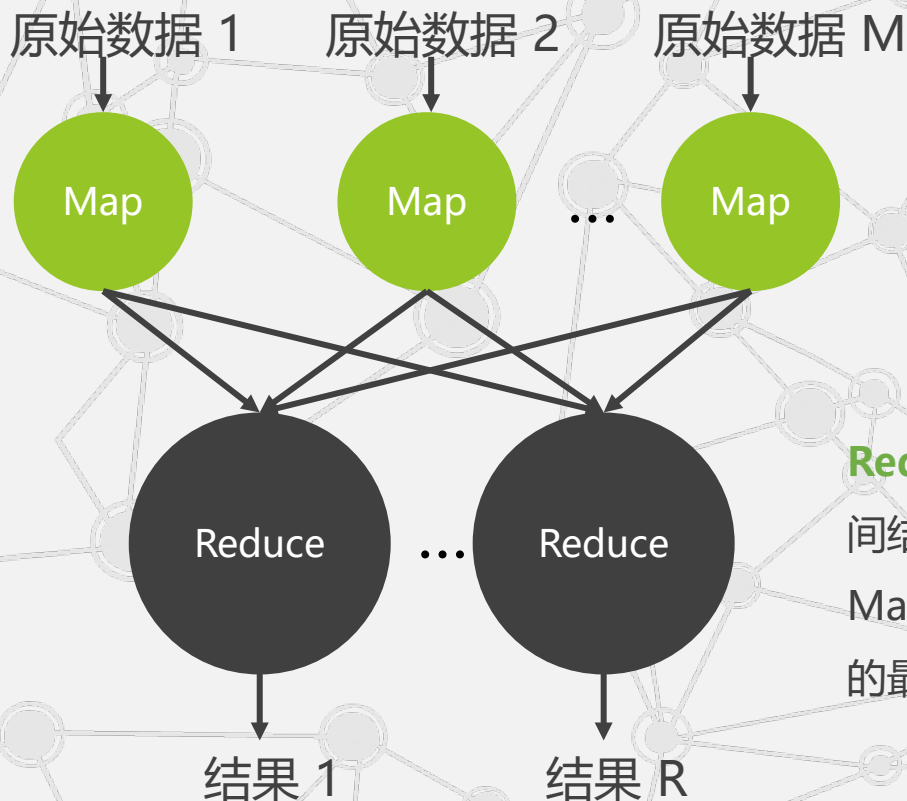
▶ 2.2.2 编程模型

2.2.3 实现机制

2.2.4 案例分析

2.2 分布式数据处理 MapReduce

● 编程模型



Map函数——对一部分原始数据进行指定的操作。每个Map操作都针对不同的原始数据，因此Map与Map之间是互相独立的，这使得它们可以充分并行化。

Reduce操作——对每个Map所产生的一部分中间结果进行合并操作，每个Reduce所处理的Map中间结果是互不交叉的，所有Reduce产生的最终结果经过简单连接就形成了完整的结果集。

2.2 分布式数据处理 MapReduce

• 编程模型

Map: $(in_key, in_value) \rightarrow \{(key_j, value_j) \mid j = 1 \dots k\}$

Reduce: $(key, [value_1, \dots, value_m]) \rightarrow (key, final_value)$

Map输入参数: in_key 和 in_value ，它指明了Map需要处理的原始数据

Map输出结果: 一组 $\langle key, value \rangle$ 对，这是经过Map操作后所产生的中间结果

Reduce输入参数: $(key, [value_1, \dots, value_m])$

Reduce工作:
对这些对应相同key的value值进行归并处理

Reduce输出结果:
 $(key, final_value)$ ，所有Reduce的结果并在一起就是最终结果

2.2 分布式数据处理 MapReduce

- 编程模型

Map: $(in_key, in_value) \rightarrow \{(key_j, value_j) \mid j = 1 \dots k\}$

Reduce: $(key, [value_1, \dots, value_m]) \rightarrow (key, final_value)$

例如，假设我们想用MapReduce来计算一个大型文本文件中各个单词出现的次数，Map的输入参数指明了需要处理哪部分数据，以“<在文本中的起始位置，需要处理的数据长度>”表示，经过Map处理，形成一批中间结果“<单词，出现次数>”。而Reduce函数处理中间结果，将相同单词出现的次数进行累加，得到每个单词总的出现次数。

2.2分布式数据处理MapReduce

2.2.1 产生背景

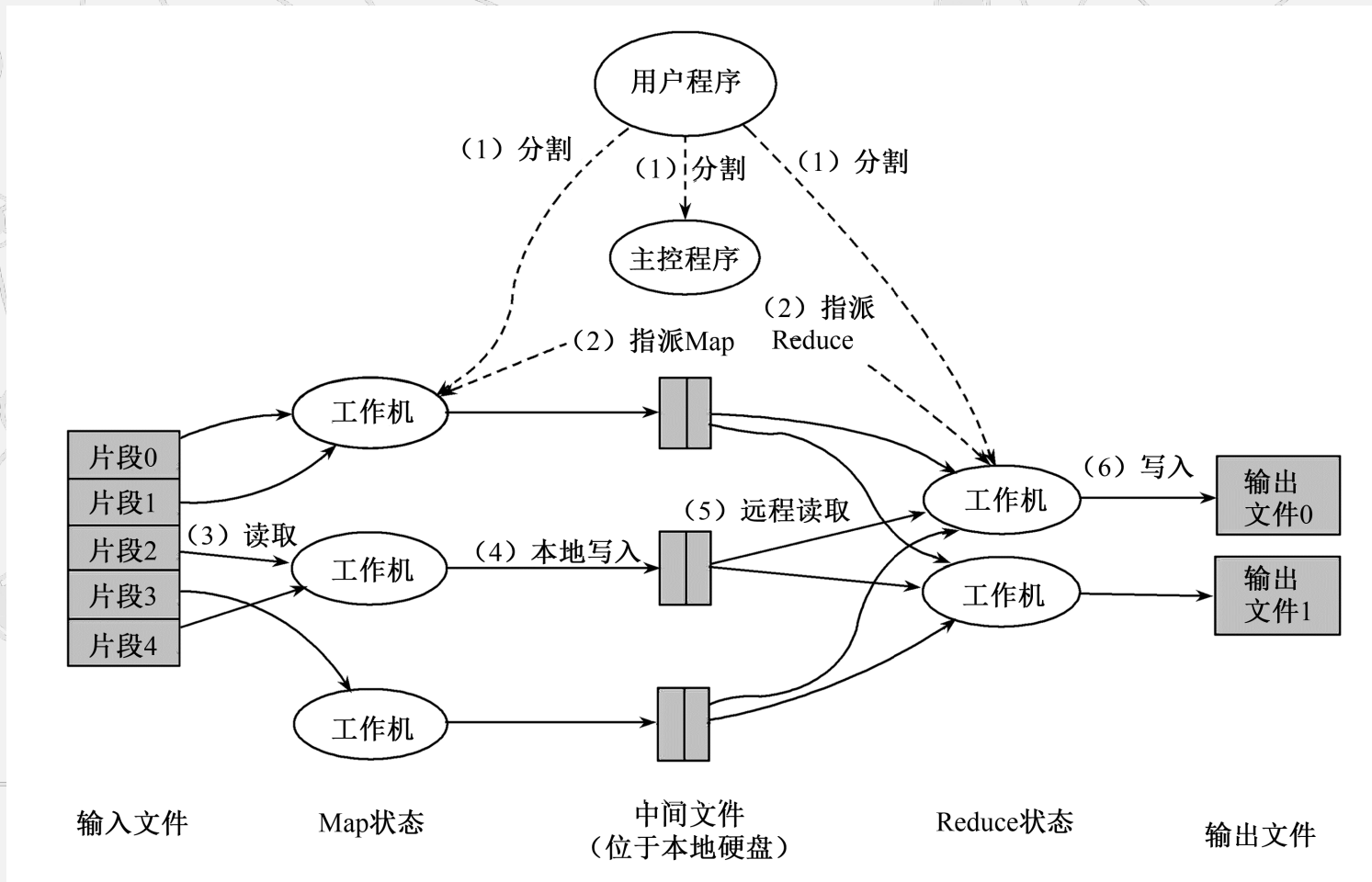
2.2.2 编程模型

► 2.2.3 实现机制

2.2.4 案例分析

2.2 分布式数据处理MapReduce

● 实现机制



2.2 分布式数据处理 MapReduce

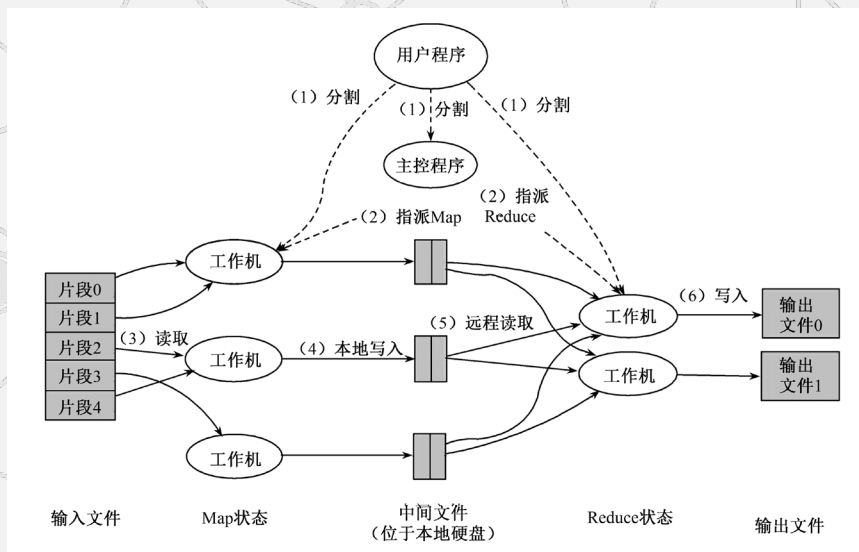
● 实现机制

(1) MapReduce函数首先把**输入文件分成M块**

每块大概16M~64MB

(可以通过参数决定),

接着在集群的机器上执行分派处理程序。



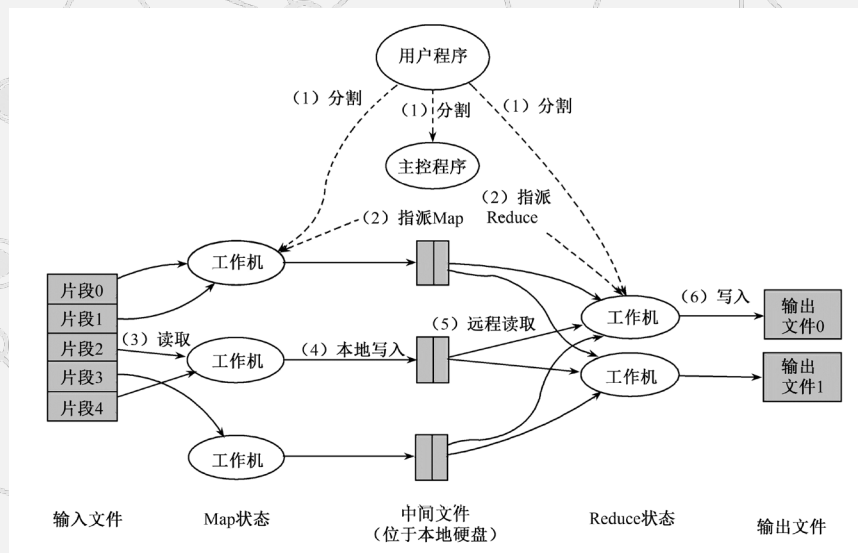
2.2 分布式数据处理MapReduce

● 实现机制

(2) 分派的执行程序中有有一个**主控程序Master**

这些分派的执行程序中有一个程序比较特别，它是主控程序Master。

剩下的执行程序都是作为Master分派工作的Worker(工作机)。总共有M个Map任务和R个Reduce任务需要分派，Master选择空闲的Worker来分配这些Map或Reduce任务。



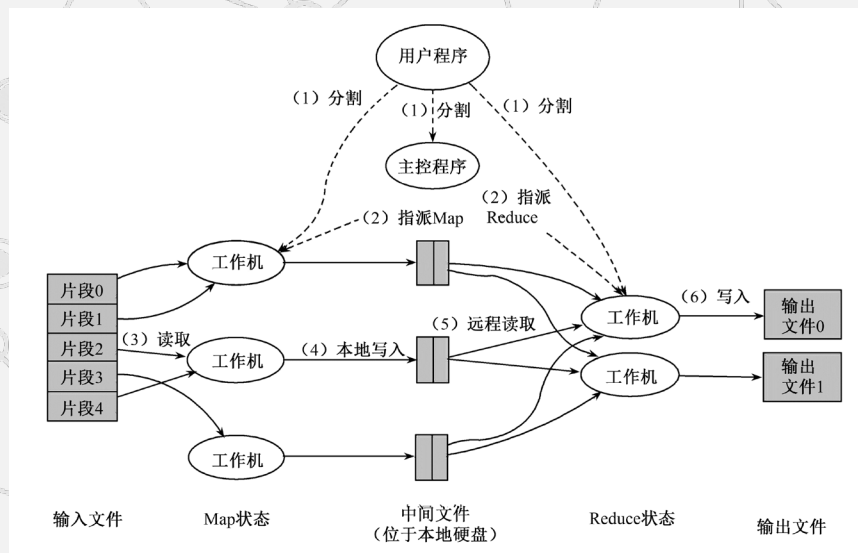
2.2 分布式数据处理 MapReduce

● 实现机制

(3) 一个被分配了Map任务的Worker读取并处理相关的输入块

它处理输入的数据，并且将分析出的
<key,value>对传递给用户定义的Map 函数。

Map函数产生的中间结果<keyvalue>
对暂时缓冲到内存。

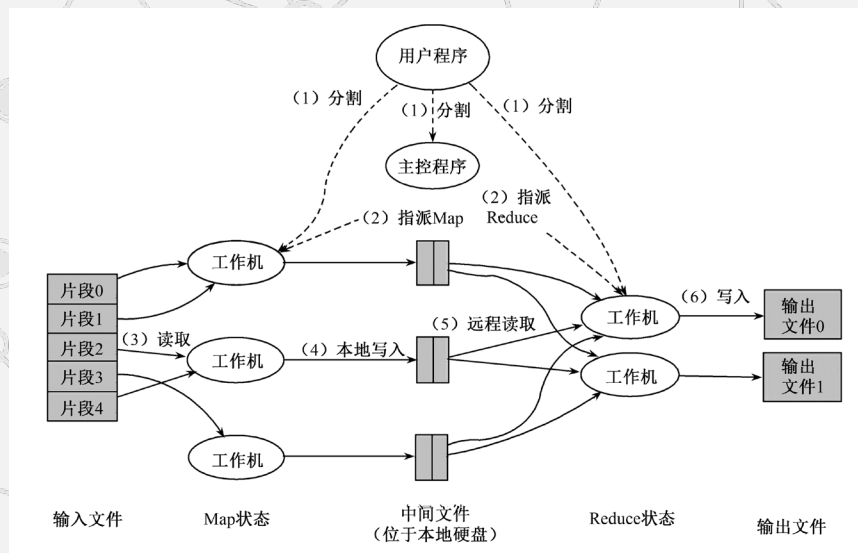


2.2 分布式数据处理 MapReduce

● 实现机制

(4) 这些缓冲到内存的中间结果将被定时写到本地硬盘，这些**数据通过分区函数分成R个区**

中间结果在本地硬盘的位置信息将被发送回Master，然后Master负责把这些位置信息传送给Reduce Worker。



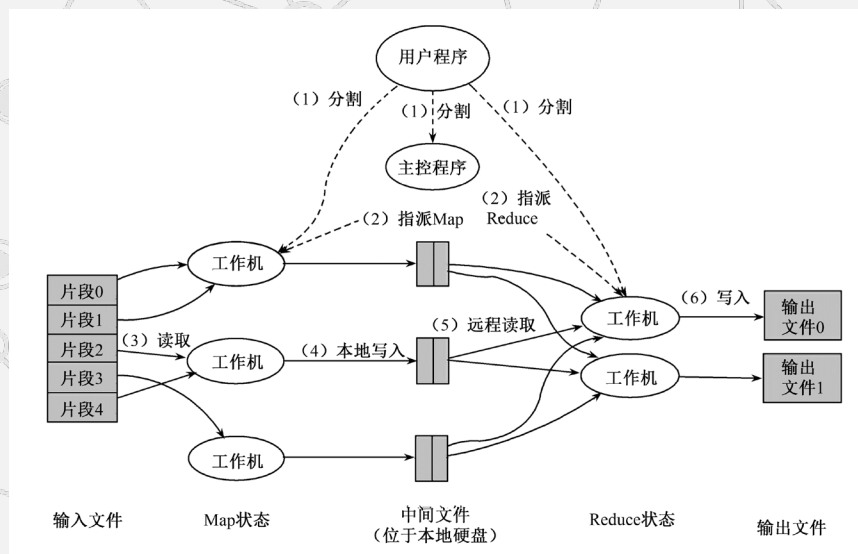
2.2 分布式数据处理 MapReduce

● 实现机制

(5) 当Master通知执行Reduce的Worker关于中间<key,value>对的位置时，它调用远程过程，从Map Worker的本地硬盘上读取缓冲的中间数据

当ReduceWorker读到所有的中间数据，它就使用中间key进行排序，这样可使相同key的值都在一起。

因为有许多不同 key的Map 都对应相同的Reduce任务，所以，排序是必需的。如果中间结果集过于庞大，那么就需要使用外排序。

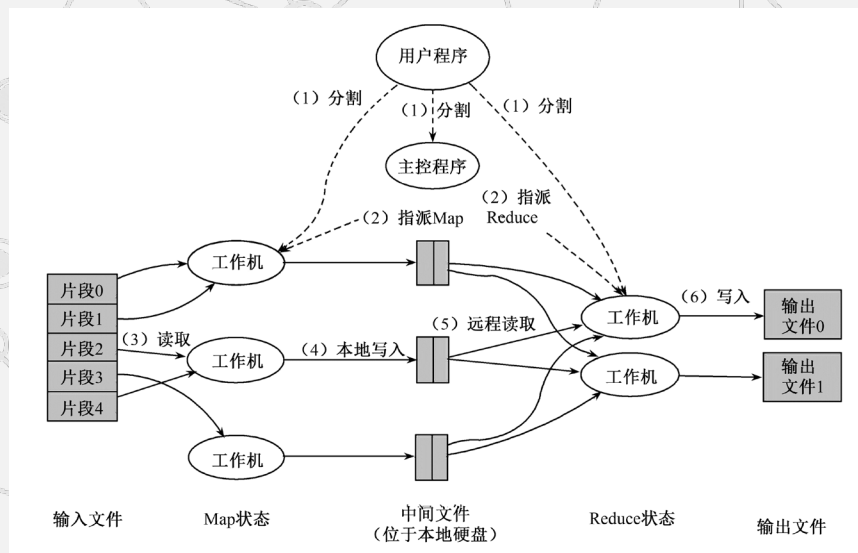


2.2 分布式数据处理 MapReduce

● 实现机制

(6) Reduce Worker根据每一个唯一中间key来遍历所有的排序后的中间数据，并且把key和相关的中间结果值集合传递给用户定义的Reduce函数

Reduce函数的结果写到一个最终的输出文件。



2.2 分布式数据处理 MapReduce

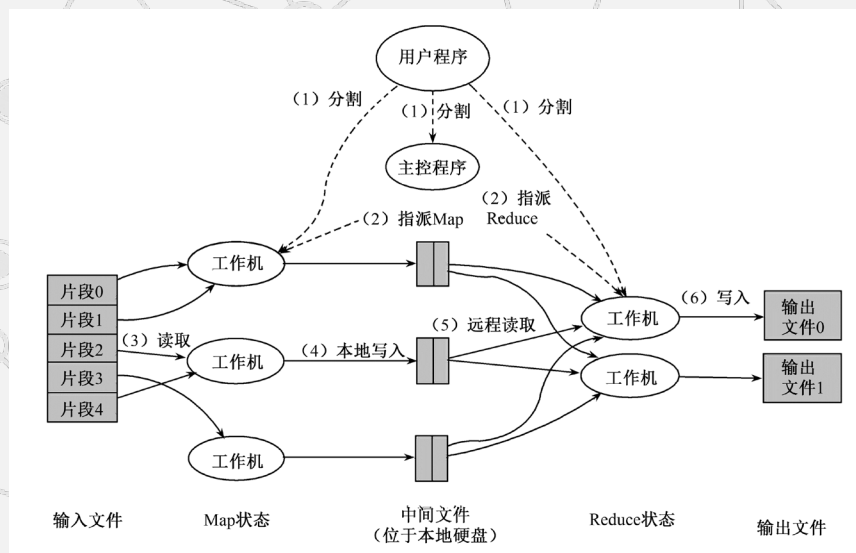
● 实现机制

(7) 当所有的Map任务和Reduce任务都完成的时候，Master激活用户程序

此时 MapReduce返回用户程序的调用点。

由于MapReduce在成百上千台机器上处理海量数据，所以容错机制是不可或缺的。

总的来说，MapReduce通过重新执行失效的地方来实现容错。



2.2 分布式数据处理 MapReduce

● 容错机制

由于MapReduce在成百上千台机器上处理海量数据，所以容错机制是不可或缺的。总的来说，MapReduce通过重新执行失效的地方来实现容错。

Master失效

Master会周期性地设置检查点（checkpoint），并导出Master的数据。一旦某个任务失效，系统就从最近的一个检查点恢复并重新执行。

由于只有一个Master在运行，如果Master失效了，则只能终止整个MapReduce程序的运行并重新开始。

Worker失效

Master会周期性地给Worker发送ping命令，如果没有Worker的应答，则Master认为Worker失效，终止对这个Worker的任务调度，把失效Worker的任务调度到其他Worker上重新执行。

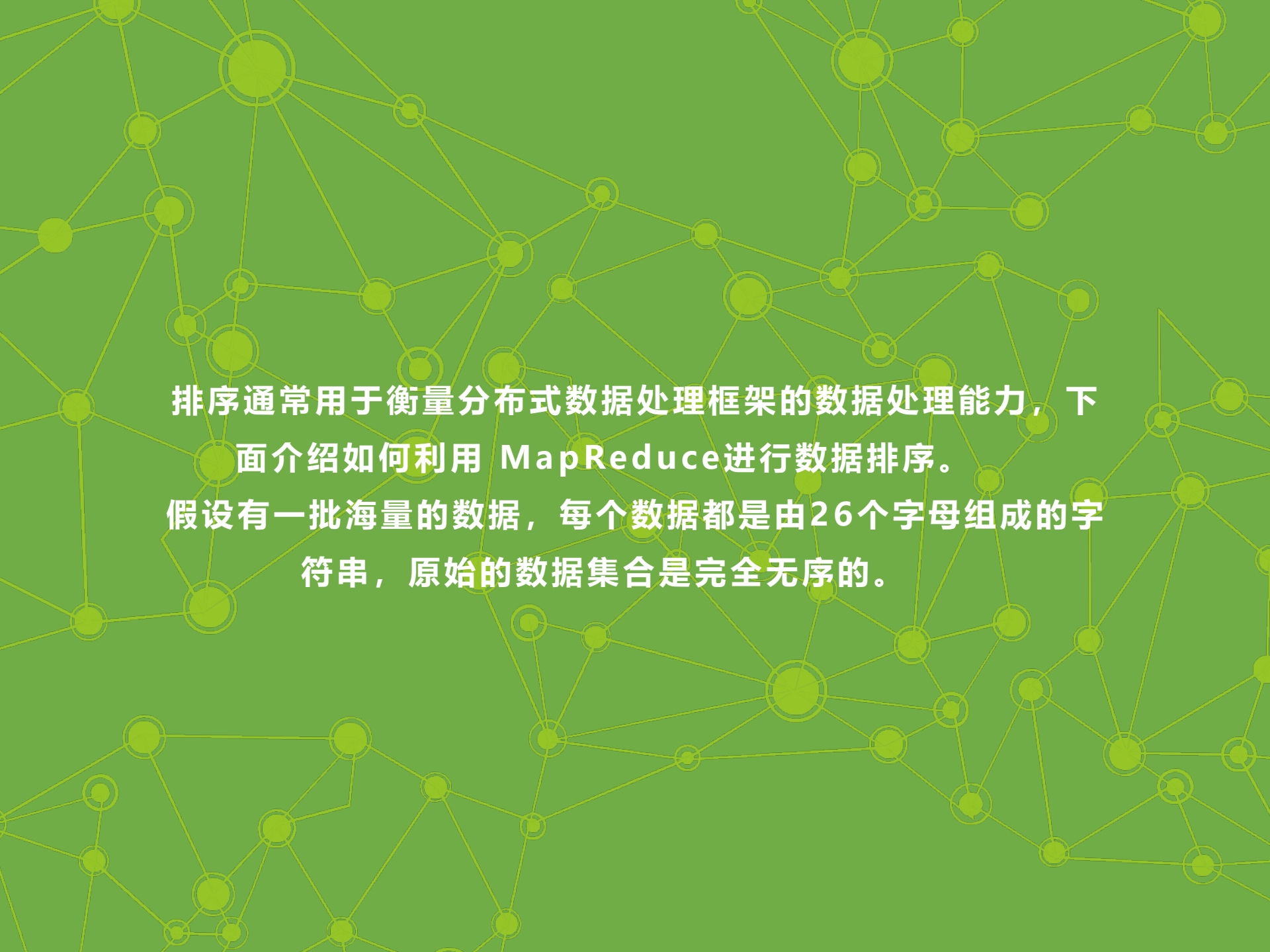
2.2 分布式数据处理MapReduce

2.2.1 产生背景

2.2.2 编程模型

2.2.3 实现机制

► 2.2.4 案例分析



排序通常用于衡量分布式数据处理框架的数据处理能力，下面介绍如何利用 MapReduce 进行数据排序。

假设有一批海量的数据，每个数据都是由26个字母组成的字符串，原始的数据集合是完全无序的。



**怎样通过MapReduce完成排序工作，
使其有序（字典序）呢？**

2.2 分布式数据处理 MapReduce

- 第一个步骤

对原始的数据进行分割 (Split) ,
得到N个不同的数据分块。

Split1: nkklacdedd
 gfgdfsdfdfd

 annnbnbvgh

Split2: dfgmdhjydf
 kghfgcxnkil

 gjghyotewgbb

.....

SplitN: hjlolsrwrbr
 hjevevxced

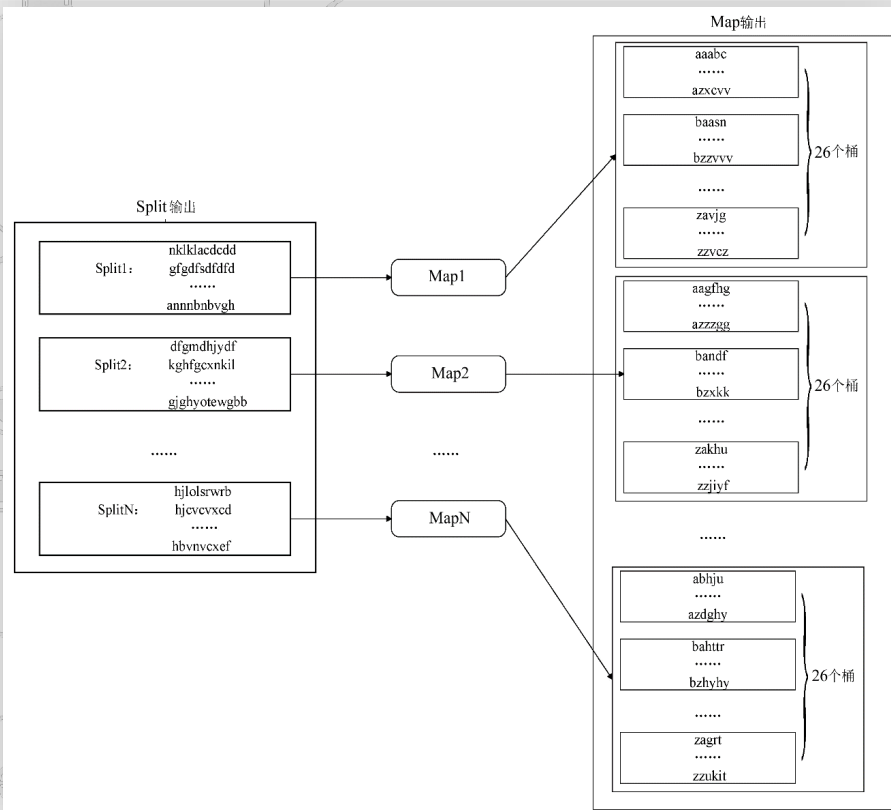
 hbnvncxef

2.2 分布式数据处理 MapReduce

• 第二个步骤

对每一个数据分块都启动一个 Map 进行处理。

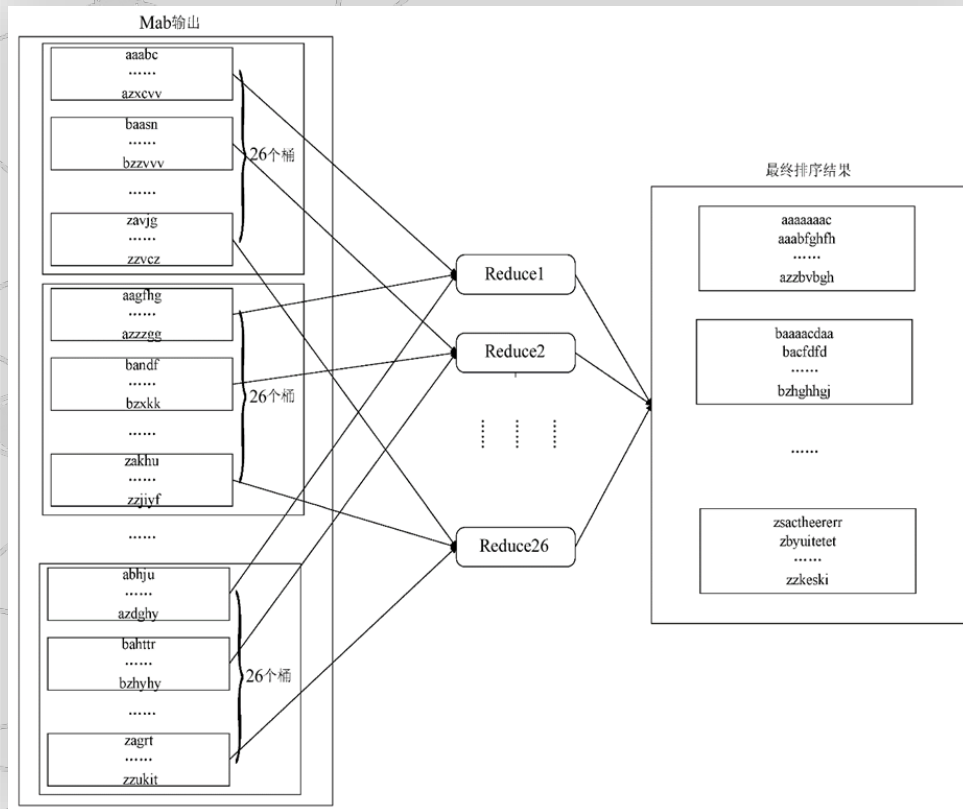
采用桶排序的方法，每个 Map 中按照首字母将字符串分配到 26 个不同的桶中。



2.2 分布式数据处理 MapReduce

• 第三个步骤

对于Map之后得到的中间结果，启动26个Reduce。按照首字母将Map中不同桶中的字符串集合放置到相应的Reduce中进行处理。





从上述过程中可以看出，
由于能够实现处理过程的完全并行化
，因此利用 MapReduce处理海量
数据是非常适合的。

2.2 分布式数据处理 MapReduce

• MapReduce思想的延续——从批处理到AI数据工程

- > - MapReduce的"分而治之"思想仍是现代大数据处理的基石
- > - Apache Spark (2014年) 继承了MapReduce思想, 但引入内存计算, 支持流处理和机器学习 (MLlib)
- > - 在LLM时代, MapReduce思想的新应用:
 - > - 大模型训练数据的预处理: 海量网页/书籍数据的清洗、去重、分词, 仍采用分布式MapReduce模式
 - > - 例如: GPT系列模型的训练数据预处理 pipeline, 每天处理数PB级文本
 - > - Hugging Face Datasets 库支持类MapReduce的分布式数据处理



本章未完待续



云计算 (第三版)

CLOUD COMPUTING Third Edition

第2章

谢谢观看